

QuickSort mit zufälligem Pivot Element



- Las-Vegas Algorithmus mit erwarteter Laufzeit $O(n \ln n)$

Beweis: Sei $(a_1, a_2, \dots, a_n)$ das Array nach dem Sortieren

Seien a_i und a_j zwei beliebige Elemente, wobei $i < j$

Sei $I_{i,j} = \begin{cases} 1 & \text{falls } a_i \text{ mit } a_j \text{ verglichen wird} \\ 0 & \text{sonst} \end{cases}$ eine Indikatorvariable

Irgendwann wird das erste mal ein Pivot $p \in \{a_i, \dots, a_j\}$ ausgewählt.

Case $p \in \{a_i, a_j\}$: a_i und a_j werden einmal verglichen

Case $p \in \{a_{i+1}, \dots, a_{j-1}\}$: a_i und a_j werden nie verglichen, weil sie a_i im linken rekursiven Aufruf ist und a_j im rechten

$(a_1, \dots, a_{i-1}, \boxed{a_i, a_{i+1}, \dots, a_{j-1}, a_j}, a_{j+1}, \dots, a_n)$
 $j-i+1$ viele Elemente

$$\mathbb{E}[I_{i,j}] = \Pr[a_i \text{ wird mit } a_j \text{ verglichen}] = \frac{2}{j-i+1}$$

$$\mathbb{E}[\text{totale Anzahl Vergleiche}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n-1} \mathbb{E}[I_{i,j}]$$

Summe über alle möglichen Paare $1 \leq i < j \leq n$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n-1} \frac{2}{j-i+1}$$

$$\mathbb{E}[I_{i,j}] = \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k}$$

Index Substitution $k = j-i+1$

$$j=i+1 \Leftrightarrow j-i+1=2 \Leftrightarrow k=2$$

$$j=n-1 \Leftrightarrow j-i+1=n-i+1 \Leftrightarrow k=n-i+1$$

$$\leq \sum_{i=1}^n \sum_{k=1}^n \frac{2}{k}$$

Summen gehen bis n

$$= 2 \cdot \sum_{i=1}^n \sum_{k=1}^n \frac{1}{k}$$

Distributivgesetz

$$= 2n \cdot H_n$$

Harmonische Reihe $H_n \stackrel{\text{def}}{=} \sum_{k=1}^n \frac{1}{k}$

$$\leq \underline{\underline{O(n \log n)}}$$

weil $H_n \leq \ln(n) + O(1)$

QuickSelect

Ziel: finde das k -kleinste Element in einem Array

QUICKSELECT(A, ℓ, r, k)

- 1: $p \leftarrow \text{Uniform}(\{\ell, \ell + 1, \dots, r\})$ ▷ wähle Pivotelement zufällig
 - 2: $t \leftarrow \text{PARTITION}(A, \ell, r, p)$
 - 3: **if** $t = \ell + k - 1$ **then** ($k-1$ Elemente sind links von t)
 - 4: **return** $A[t]$ ▷ gesuchtes Element ist gefunden
 - 5: **else if** $t > \ell + k - 1$ **then**
 - 6: **return** QUICKSELECT($A, \ell, t - 1, k$) ▷ gesuchtes Element ist links
 - 7: **else**
 - 8: **return** QUICKSELECT($A, t + 1, r, k - t$) ▷ gesuchtes Element ist rechts
-

Erwartete Laufzeit: $O(n)$

(kam meines Wissens nach noch nie bei einer Prüfung)

Duplikate finden mit direktem Sortieren

Universum: U ist die Menge aller möglichen Elementen

Datensatz: $D = (s_1, s_2, \dots, s_n)$ ist eine Folge von n Elementen aus einem Universum U

Duplikat: Ein Paar (i, j) sodass $s_i = s_j$, wobei $i < j$

1.) Sortieren

2.) Liste durchlaufen und dabei alle Duplikate ausgeben

Bsp $(4, 19, 1, 2, 4, 5, 2, 4)$) Sortieren
 Index: $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{matrix}$
 $(1, \underline{2}, \underline{2}, \underline{4}, \underline{4}, \underline{4}, 5, 19)$

Duplikate: $(2, 3), (4, 5), (4, 6), (5, 6)$

Laufzeit: $O(\underbrace{n \log n}_{\text{Sortieren}} + \underbrace{|\text{Duplikate}|}_{\text{Duplikate ausgeben}})$

Wie viele Duplikate gibt es höchstens bei n Elementen?

Duplikate finden mit Hashfunktion

Motivation: Vergleiche von "schwierigen" Elementen (z.B. Graphen, DNA-Strings) können teuer sein

Hashfunktion: $h: U \rightarrow \{1, \dots, m\}$ berechnet für ein Element $u \in U$ den Hashwert $h(u)$

- m soll viel kleiner als $|U|$ sein
- h soll effizient berechenbar sein
- h soll zufällig gleichverteilt sein. $\forall u \in U \forall i \in [m]: \Pr[h(u) = i] = \frac{1}{m}$
- h hat die Eigenschaft: $s_i = s_j \Rightarrow h(s_i) = h(s_j)$

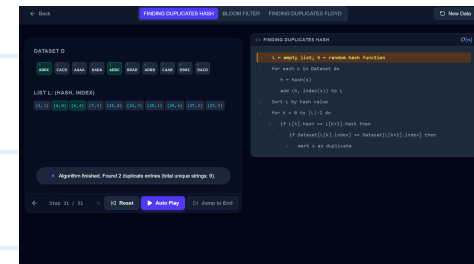
Kollisionen: sind Paare (i, j) mit $s_i \neq s_j$ aber $h(s_i) = h(s_j)$

1.) Hashen und Index merken

2.) Sortieren nach Hashwert aufsteigend

3.) Liste durchlaufen und dabei alle Hashwert-Duplikate

überprüfen und ausgeben, wenn es keine Kollisionen sind



Laufzeit: $O(n \log n + |\text{Duplikate}| + |\text{Kollisionen}|)$

Hashen und Sortieren Duplikate ausgeben Kollisionen überprüfen

Für $m = n^2$: ist die erwartete Anzahl Kollisionen konstant

Beweis: Sei $K_{i,j}$ die Indikatorvariable für das Ereignis " (i, j) ist eine Kollision"

$$\Rightarrow \Pr[K_{i,j} = 1] = \begin{cases} \frac{1}{m} & \text{falls } s_i \neq s_j \\ 0 & \text{sonst} \end{cases} \quad (\text{weil } s_i = s_j \text{ ein Duplikat ist, und daher keine Kollision sein kann})$$

$$\Rightarrow \mathbb{E}[K_{i,j}] \leq \frac{1}{m}$$

$$\mathbb{E}[\text{Anzahl Kollisionen}] = \sum_{1 \leq i < j \leq n} \mathbb{E}[K_{i,j}]$$

$$\mathbb{E}[K_{i,j}] \leq \frac{1}{m}$$

$$\leq \sum_{1 \leq i < j \leq n} \frac{1}{m}$$

$$\binom{n}{2} = \text{Anzahl Paare } (i,j) \text{ mit } i < j$$

$$= \binom{n}{2} \cdot \frac{1}{m}$$

$$\text{def. Binomialkoeffizient } \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$= \frac{n(n-1)}{2} \cdot \frac{1}{m}$$

$$\text{ausmultiplizieren}$$

$$= \frac{n^2 - n}{2} \cdot \frac{1}{m}$$

$$\text{Annahme: } m = n^2$$

$$= \frac{n^2 - n}{2} \cdot \frac{1}{n^2}$$

$$\text{dividieren}$$

$$= \frac{1 - \frac{1}{n}}{2}$$

$$= \frac{1}{2} - \frac{1}{2n}$$

$$< \frac{1}{2} \Rightarrow \text{Konstant}$$

Duplikate finden mit Hase-Igel-Algorithmus (Floyd)

Gegeben: ein Array $a[1..n]$ wobei alle Elemente zwischen 1 und $n-1$ liegen

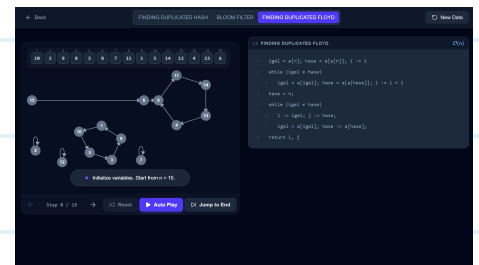
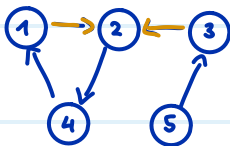
Gesucht: ein Duplikat, also zwei verschiedene Indices i, j sodass $a[i] = a[j]$

(existiert immer wegen Pigeonhole-Prinzip)

Umformulierung als Graph: Knoten $V = \{1, \dots, n\}$ und Kanten $\{(i, a[i]) \mid 1 \leq i \leq n\}$

neues Ziel: finde einen Knoten mit zwei eingehenden Kanten

Bsp. $A = (\boxed{2}, 4, \boxed{2}, 1, 3)$

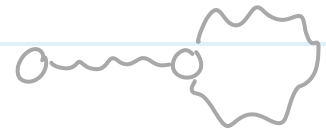


Subgraph: Wir betrachten nur noch alle von n aus erreichbaren Knoten.

Jeder Knoten hat genau eine ausgehende Kante (nämlich von i zu $a[i]$)

Der Subgraph muss ein Pfad der Länge $k \geq 1$ sein,

gefolgt von einem Kreis der Länge $\ell \geq 1$



Knotenfolge: $x_0 = n$, und rekursiv $x_{t+1} = a[x_t]$ für alle $t \geq 0$

Bsp $A = (3, 6, 4, 2, 5, 4, 1)$

1 Treffen: Es gibt ein $t \in \{1, \dots, n\}$ sodass $x_t = x_{2t}$

(also wenn Hase und Igel mit Geschwindigkeit 2 (Hase) und 1 (Igel) bei

Knoten n loslaufen, treffen sie sich spätestens nach n Schritten)

igel = $a[n]$; hase := $a[a[n]]$; $i := 1$

while (igel $\neq$ hase)

 igel = $a[\text{igel}]$; hase := $a[a[\text{hase}]]$; $i := i + 1$

1 Schritt

2 Schritte

Beweis: Sei r der "Rest" den wir auffüllen müssen, sodass $k+r$ das kleinste Vielfache

von der Kreislänge ℓ ist: $k+r = \underbrace{\left\lceil \frac{k}{\ell} \right\rceil}_{\text{Anzahl angefangene male, wie oft } \ell \text{ in } k \text{ hinein passt}} \cdot \ell \Leftrightarrow r = \left\lceil \frac{k}{\ell} \right\rceil \cdot \ell - k$

Anzahl angefangene male, wie oft ℓ in k hinein passt

$\Rightarrow$ Dann liegt x_{k+r} auf dem Kreis (weil der Kreis bei k beginnt und $k+r \geq k$)

$\Rightarrow x_{k+r} = x_{2(k+r)}$ weil $k+r$ ein Vielfaches der Kreislänge ist. Also $x_{2(k+r)}$ macht einfach einige Kreisumrundungen mehr)

$\Rightarrow$ wir wählen also $t = k+r$

Und es gilt: $t = k+r$

$$= \left\lceil \frac{k}{\ell} \right\rceil \ell$$

per Definition

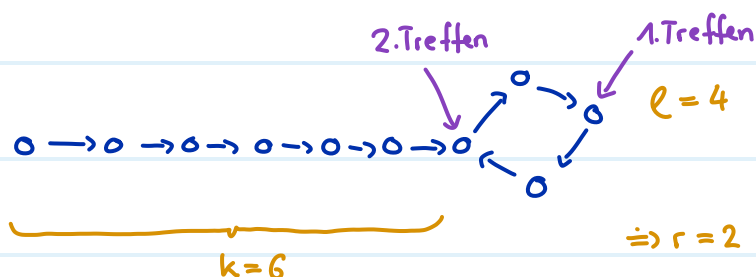
$$< \frac{k}{\ell} \ell + \ell$$

auf runden

$$= k + \ell$$

Pfadlänge + Kreislänge $\leq n$ weil es nur n Kanten gibt

$$\leq n$$



2 Treffen: es gilt: $\underbrace{x_k}_{\text{Hase}} = \underbrace{x_{k+t}}_{\text{Igel}}$

(der Hase startet noch einmal bei Knoten n und hat nun Geschwindigkeit 1. Er wird den Igel genau nach k Schritten treffen, also beim Anfang des Kreises)

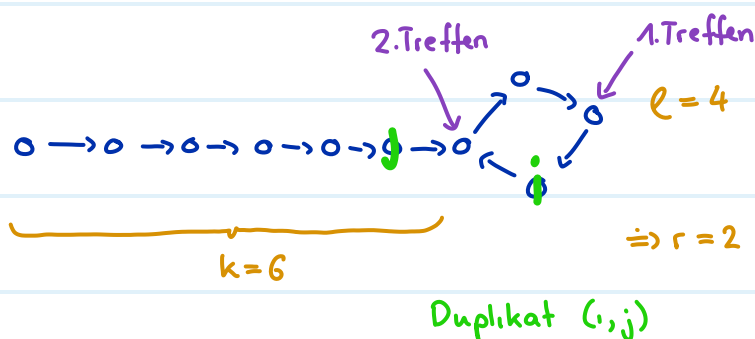
```

hase = n;   Hase wird zurück an den Startknoten n gestellt
while (igel ≠ hase)
    i := igel; j := hase;
    igel = a[igel]; hase := a[hase];
return i, j
           1 Schritt           1 Schritt
  
```

Beweis: $x_{k+t} = x_k$ weil x_k auf dem Kreis liegt (x_k ist per Definition der erste Knoten des Kreises)
und t ein Vielfaches der Kreislänge ℓ ist (nach jeder Kreisumdrehung sind wir wieder bei x_k)

Laufzeit: $O(n)$

Zusätzlicher Speicher: 4 Variablen



Wahrscheinlichkeits - DP

Du befindest dich in einem Casino an einem seltsamen Spieltisch. Du darfst eine gezinkte Münze werfen. Diese Münze fällt zu **70 % auf Kopf** und zu **30 % auf Zahl**.

Der Croupier macht dir folgendes Angebot: Du darfst die Münze genau N -mal werfen. Du gewinnst den Hauptpreis, sobald du **dreimal direkt hintereinander "Kopf"** wirfst (ein sogenannter 3er-Streak). Sobald du das schaffst, ist das Spiel sofort gewonnen, egal wie viele Würfe du noch übrig hättest.

Dimension: $DP[0, \dots, N][0, \dots, 3]$

Teilproblem: $DP[i, s] =$

Base Case: $DP[0, s] =$

$DP[i, 3] =$

Rekursion: $DP[i, s] =$

Reihenfolge:

Lösung:

Laufzeit: